

1. Creating storage account

The screenshot shows the Microsoft Azure portal interface. The browser address bar displays the URL: `https://portal.azure.com/#@michaelcloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...`. The page title is "ststoraeg Storage account". The left sidebar contains a navigation menu with options like Overview, Activity log, Tags, Diagnose and solve problems, Access Control (IAM), Data migration, Events, Storage browser, Partner solutions, Resource visualizer, Data storage, Containers, File shares, Queues, and Tables. The main content area shows the "Overview" tab for the storage account. It includes a search bar, a list of actions (Upload, Open in Explorer, Delete, Move, Refresh, Open in mobile, CLI / PS, Feedback), and a "JSON View" link. The "Essentials" section displays key information: Resource group (LabsKraft2027), Location (southindia), Subscription (LabsKraft), Subscription ID (d008c7cc-9795-4292-bf79-74ae52cda63d), Disk state (Available), and Tags (Add tags). The "Properties" section shows the account kind as "StorageV2 (general purpose v2)" and the provisioning state as "Succeeded". The "Data Lake Storage" section indicates it is "Enabled". The "Security" section shows "Require secure transfer for REST API operations" is "Enabled". The bottom status bar shows the time as 15:37 on 25-08-2025.

2. Creating container

The screenshot shows the Microsoft Azure portal interface, specifically the "Containers" page for the storage account "ststoraeg". The browser address bar displays the URL: `https://portal.azure.com/#@michaelcloudkraft@hotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...`. The page title is "ststoraeg | Containers Storage account". The left sidebar is the same as in the first screenshot, but the "Containers" option is selected. The main content area shows a table of containers. At the top, there are buttons for "Add container", "Upload", "Refresh", "Delete", "Change access level", "Restore containers", and "Edit columns". A search bar labeled "Search containers by prefix" is present, along with a dropdown menu set to "Only show active containers". The table lists two containers: "slogs" and "raw".

☐	Name	Last modified	Anonymous access level	Lease state	
☐	slogs	25/8/2025, 10:19:55 am	Private	Available	...
☐	raw	25/8/2025, 10:23:17 am	Private	Available	...

The bottom status bar shows the time as 15:37 on 25-08-2025.

3. Adding data file to container

The screenshot shows the Microsoft Azure portal interface for a storage account. The breadcrumb navigation indicates the path: Home > ststorageeg | Containers >. The selected container is named 'raw'. The left sidebar shows the 'Overview' tab selected, with links to 'Diagnose and solve problems', 'Access Control (IAM)', and 'Settings'. The main content area displays the container's overview, including the authentication method (Access key) and a search bar for blobs. Below this, a table lists the contents of the container, showing two items: '__unitystorage' and 'retail_iot_data.csv'.

☐	Name	Last modified	Access tier	Blob type	Size	Lease state
☐	__unitystorage	25/8/2025, 11:08:18 am				...
☐	retail_iot_data.csv	25/8/2025, 10:23:43 am	Hot (Inferred)	Block blob	130.76 KiB	Available

4. Creating access connector for databricks while creating new external location for storage credential for creating new Unity Catalog in databricks

workspace

The screenshot shows the Microsoft Azure portal interface. The browser address bar displays the URL: `https://portal.azure.com/#@michaelkloudkrafthotmail.onmicrosoft.com/resource/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resour...`. The page title is "st-connect" and the subtitle is "Access Connector for Azure Databricks". The left sidebar contains a navigation menu with options: Overview, Activity log, Access control (IAM), Tags, Resource visualizer, Settings, Automation, and Help. The main content area is titled "Essentials" and displays the following information:

- Resource group: [LabsKraft2027](#)
- Location: Southeast Asia
- Subscription: [LabsKraft](#)
- Subscription ID: d008c7cc-9795-4292-bf79-74ae52cda63d
- Tags: [\(edit\)](#), [Add tags](#)
- State: Succeeded
- Resource ID: /subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

A "JSON View" link is visible in the top right corner of the Essentials section. The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 15:39 on 25-08-2025.

5. Creating Unity Catalog with required schemas for Bronze, Silver, Gold layers

The screenshot displays the Databricks Unity Catalog interface. The browser address bar shows the URL: `https://adb-1742522860829605.5.azuredatabricks.net/explore/data/st_capstone_retail?o=1742522860829605`. The interface includes a left sidebar with navigation options: New, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Marketplace, SQL, SQL Editor, Queries, Dashboards, Genie, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, and AI/ML. The main content area is titled "Catalog Explorer" and shows the "st_capstone_retail" catalog. The "Overview" tab is selected, displaying a description and a table of schemas.

Description

AI generate Add

Filter schemas 5 schemas

Name	Owner	Created at
bronze	tredence20@michaelkloudkraf...	Aug 25, 2025, 10:49 AM
default	tredence20@michaelkloudkraf...	Aug 25, 2025, 10:36 AM
gold	tredence20@michaelkloudkraf...	Aug 25, 2025, 01:57 PM
information_schema	System user	Aug 25, 2025, 10:36 AM
silver	tredence20@michaelkloudkraf...	Aug 25, 2025, 11:46 AM

About this catalog

Owner: Tredence LabsKraft20

Tags

Add tags

The bottom of the screen shows the Windows taskbar with various application icons and the system clock indicating 15:38 on 25-08-2025.

6. Bronze layer code(notebook)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("BronzeIngestionADLS").getOrCreate()

# -----
# 1. Source ADLS path (replace with your container + storage account)
# -----
source_path =
"abfss://raw@ststoraeg.dfs.core.windows.net/retail_iot_data.csv"

# -----
# 2. Read raw files (CSV + JSON supported)
# -----
df_raw = spark.read.format("csv") \
    .option("header", True) \
    .option("inferSchema", True) \
    .load(source_path)

# 3. Write to Bronze Delta Table in Unity Catalog
# -----
df_raw.write.format("delta") \
    .mode("append") \
    .saveAsTable("st_capstone_retail.bronze.sales_bronze")

# -----
# 4. Validate ingestion
# -----
display(spark.table("st_capstone_retail.bronze.sales_bronze"))
```

7. Silver Layer code(notebook)

```
from pyspark.sql import functions as F

# -----
# 1. Load Bronze Delta Tables (from ADLS ingestion step)
# -----

bronze_df = spark.table("st_capstone_retail.bronze.sales_bronze")

# -----
# 2. Data Cleaning & Deduplication
# -----
# - Standardize column names
# - Handle nulls
# - Remove duplicates
# -----

clean_df = (bronze_df
    .dropDuplicates(["OrderID", "CustomerID"])
    .fillna({"AnomalyType": "No Anomaly"})
    .withColumn("sale_date", F.to_date("OrderTime"))
    .withColumn("sale_month", F.date_format("OrderTime", "yyyy-MM")) #
monthly aggregation
)

# -----
# 4. Write to Silver Delta Table
# -----

clean_df.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
```

```

        .saveAsTable("st_capstone_retail.silver.sales_silver")

# -----
# 5. Verify
# -----

display(spark.table("st_capstone_retail.silver.sales_silver").limit(20))

```

8. Gold Layer Code(notebook)

```

from pyspark.sql import functions as F
from pyspark.sql.window import Window

# -----
# Load Silver Table
# -----

silver_df = spark.table("st_capstone_retail.silver.sales_silver")

# Window spec for store-level calculations
window_store = Window.partitionBy("StoreID")

# -----
# 1. Daily Sales per Region
# -----

daily_sales = (silver_df
    .groupBy("Region", "sale_date")
    .agg(F.sum("SalesAmount").alias("daily_sales"))
)

daily_sales.write.format("delta") \
    .mode("overwrite") \

```

```

.option("overwriteSchema", "true") \
.saveAsTable("st_capstone_retail.gold.daily_sales_per_region")

# -----
# 2. Monthly Sales per Region
# -----
monthly_sales = (silver_df
  .groupBy("Region", "sale_month")
  .agg(F.sum("SalesAmount").alias("monthly_sales"))
)

monthly_sales.write.format("delta") \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .saveAsTable("st_capstone_retail.gold.monthly_sales_per_region")

# -----
# 3. Device Anomaly Trend per Store
# -----
anomaly_trend = (silver_df
  .groupBy("StoreID", "sale_date")
  .agg(F.sum("AnomalyFlag").alias("anomaly_count"))
)

anomaly_trend.write.format("delta") \
  .mode("overwrite") \
  .option("overwriteSchema", "true") \
  .saveAsTable("st_capstone_retail.gold.anomaly_trend_per_store")

# -----

```

4. Conversion Impact per Store

```
conversion_impact = (silver_df
    .groupBy("StoreID")
    .agg(
        F.sum("SalesAmount").alias("total_sales"),
        F.sum("AnomalyFlag").alias("failed_device_count")
    )
    .withColumn(
        "conversion_impact_pct",
        F.col("failed_device_count") /
        F.sum("failed_device_count").over(window_store) * 100
    )
)

conversion_impact.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("st_capstone_retail.gold.conversion_impact")
```

5. Store Tiers

```
store_tiers = (silver_df
    .groupBy("StoreID")
    .agg(F.sum("SalesAmount").alias("total_sales"))
    .withColumn("tier",
        F.when(F.col("total_sales") >= 100000, "High")
        .when(F.col("total_sales") >= 50000, "Medium")
        .otherwise("Low")
    )
)
```

```

)
)

store_tiers.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("st_capstone_retail.gold.store_tiers")

# -----
# 6. Weighted Avg Sales vs Anomalies
# -----

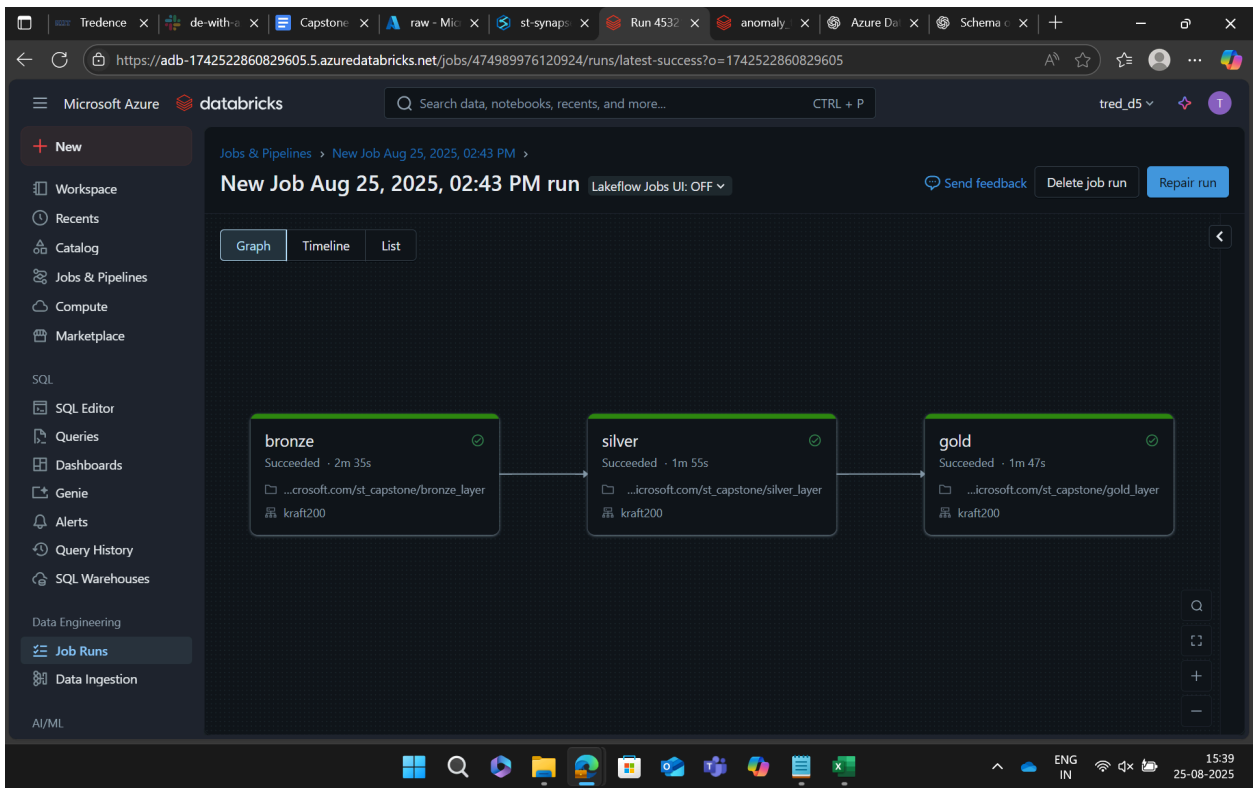
weighted_avg = silver_df.withColumn(
    "weighted_sales_vs_anomalies",
    F.sum(F.col("SalesAmount") * F.col("AnomalyFlag")).over(window_store) /
    F.count("SalesAmount").over(window_store)
).select("StoreID", "weighted_sales_vs_anomalies").distinct()

weighted_avg.write.format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("st_capstone_retail.gold.weighted_avg_sales_vs_anomalies")

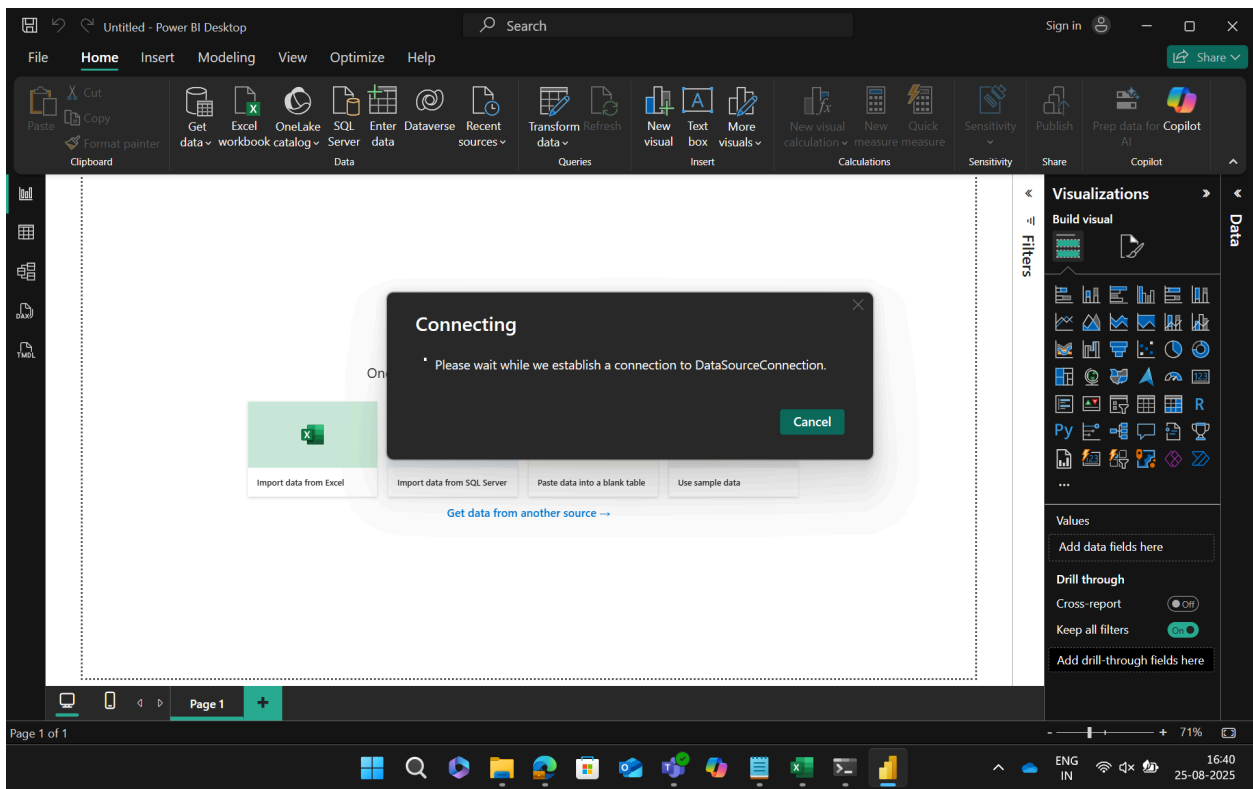
display(spark.table("st_capstone_retail.gold.daily_sales_per_region"))
display(spark.table("st_capstone_retail.gold.monthly_sales_per_region"))

```

9. Execution of all 3 layers as a single Job Pipeline



10. Connecting Unity Catalog to Power BI



11. Creating Azure Event Hub namespace and Event to capture continuous data from streaming

The screenshot displays the Microsoft Azure portal interface. The top navigation bar includes the 'Microsoft Azure' logo, a search bar, and the user profile 'tredence20@michaelklo...'. The main content area is titled 'stnamespace | Overview' and shows a deployment in progress. The deployment details include the name 'stnamespace', subscription 'LabsKraft', and resource group 'LabsKraft2027'. The start time is '8/25/2025, 3:40:01 PM' and the correlation ID is '79419b43-ea9e-4d46-b077-08...'. A table with columns 'Resource', 'Type', 'Status', and 'Operatic' is shown, but it is empty with the message 'There are no resources to display.' On the right side, there are links for 'Microsoft Defender for Cloud', 'Free Microsoft tutorials', and 'Work with an expert'.

Home > **stnamespace** | Overview

Deployment

Search resources, services, and docs (G+)

Microsoft Defender for Cloud

Secure your apps and infrastructure

Go to Microsoft Defender for Cloud >

Free Microsoft tutorials

Start learning today >

Work with an expert

Azure experts are service provider partners who can help manage your assets on Azure and be your first line of support.

Find an Azure expert >

Deployment is in progress

Deployment name : stnamespace
Subscription : LabsKraft
Resource group : LabsKraft2027

Start time : 8/25/2025, 3:40:01 PM
Correlation ID : 79419b43-ea9e-4d46-b077-08...

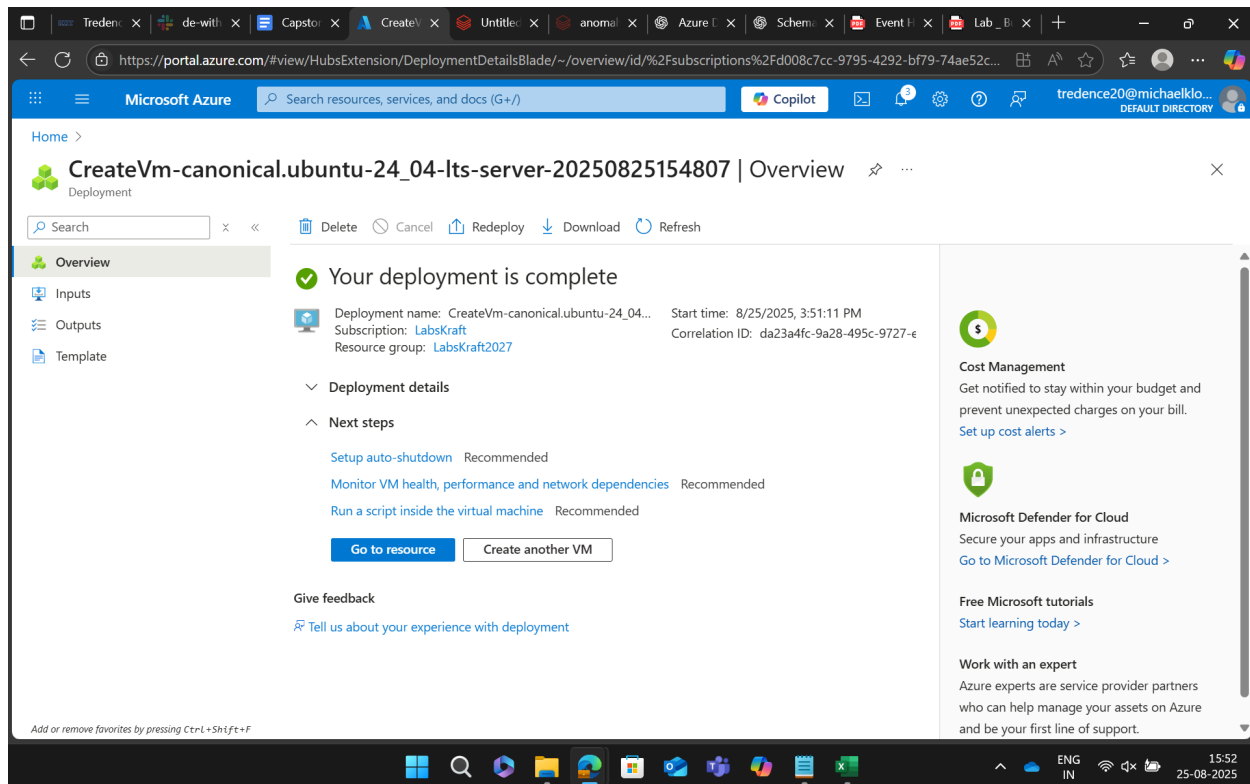
Deployment details

Resource	Type	Status	Operatic
There are no resources to display.			

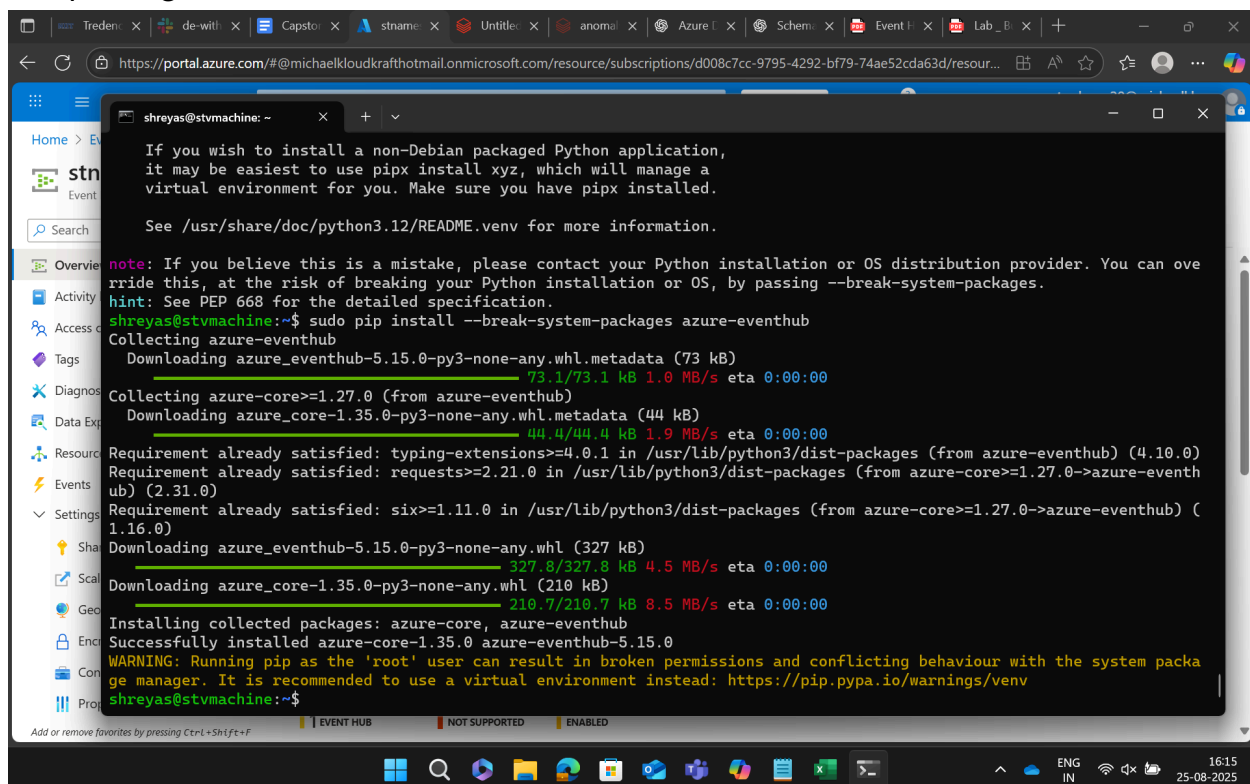
Add or remove favorites by pressing Ctrl+Shift+F

15:40 25-08-2025

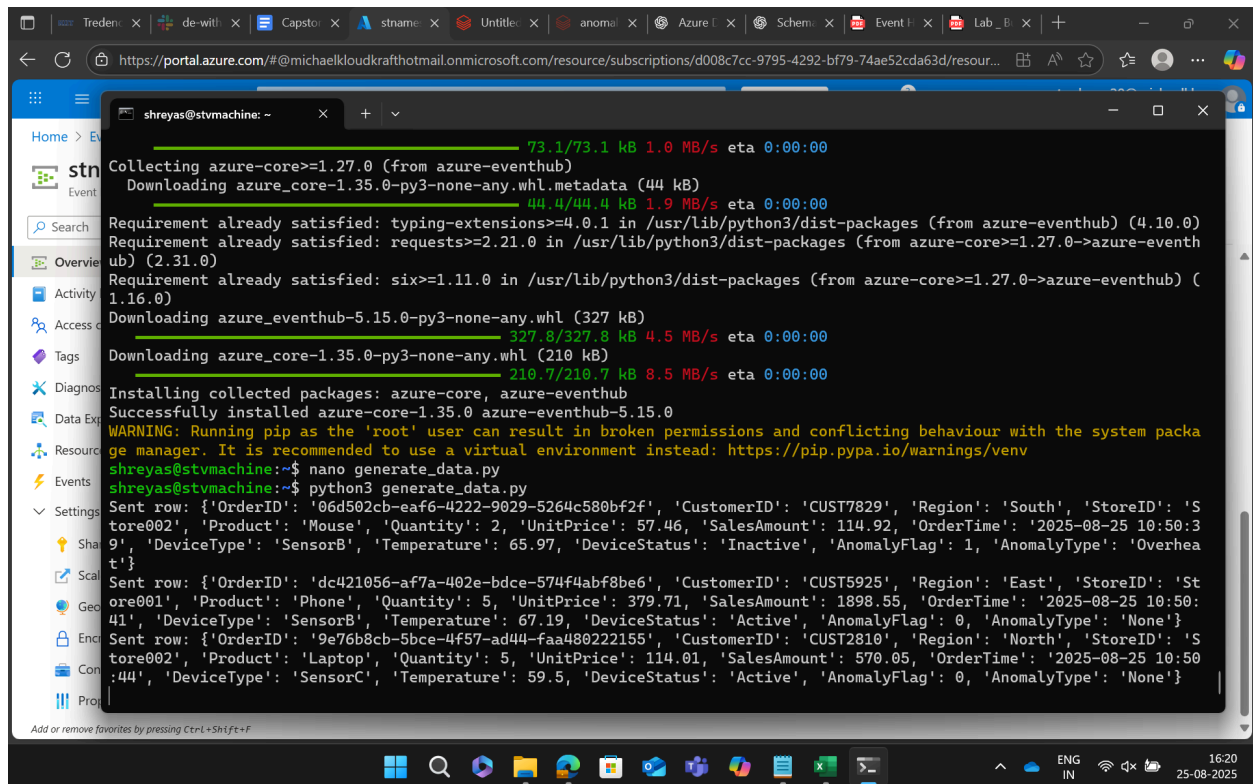
12. Provisioning resources for Virtual Machine which will send streaming data



13. Connecting to VM on cmd using authentication and installing necessary libraries and packages

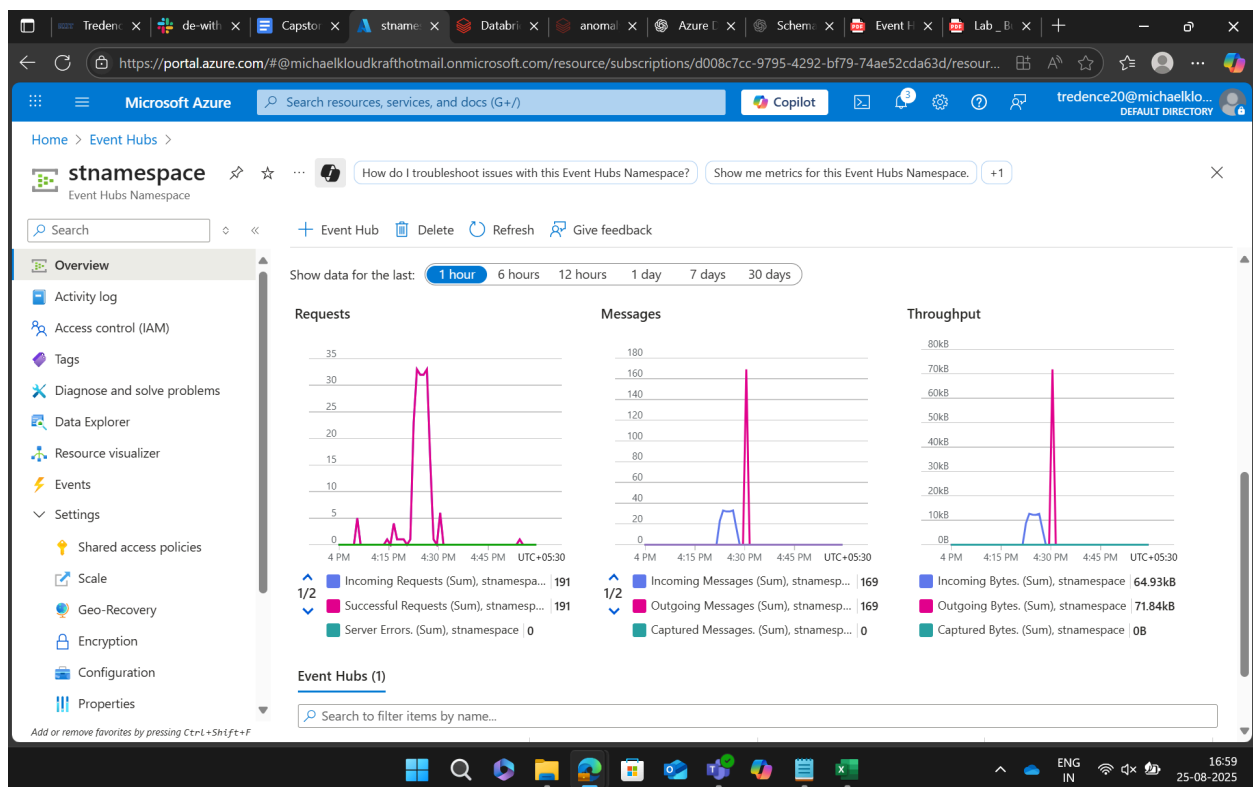


14. Creating python file to send streaming data and then running it on VM



```
shreyas@stvmachine: ~  
Collecting azure-core>=1.27.0 (from azure-eventhub)  
  Downloading azure_core-1.35.0-py3-none-any.whl.metadata (44 kB)  
    73.1/73.1 kB 1.0 MB/s eta 0:00:00  
Requirement already satisfied: typing-extensions>=4.0.1 in /usr/lib/python3/dist-packages (from azure-core>=1.27.0) (4.10.0)  
Requirement already satisfied: requests>=2.21.0 in /usr/lib/python3/dist-packages (from azure-core>=1.27.0->azure-eventhub) (2.31.0)  
Requirement already satisfied: six>=1.11.0 in /usr/lib/python3/dist-packages (from azure-core>=1.27.0->azure-eventhub) (1.16.0)  
  Downloading azure_eventhub-5.15.0-py3-none-any.whl (327 kB)  
    327.8/327.8 kB 4.5 MB/s eta 0:00:00  
  Downloading azure_core-1.35.0-py3-none-any.whl (210 kB)  
    210.7/210.7 kB 8.5 MB/s eta 0:00:00  
Installing collected packages: azure-core, azure-eventhub  
Successfully installed azure-core-1.35.0 azure-eventhub-5.15.0  
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv  
shreyas@stvmachine:~$ nano generate_data.py  
shreyas@stvmachine:~$ python3 generate_data.py  
Sent row: {'OrderID': '06d502cb-eaf6-4222-9029-5264c580bf2f', 'CustomerID': 'CUST7829', 'Region': 'South', 'StoreID': 'Store002', 'Product': 'Phone', 'Quantity': 2, 'UnitPrice': 57.46, 'SalesAmount': 114.92, 'OrderTime': '2025-08-25 10:50:39', 'DeviceType': 'SensorB', 'Temperature': 65.97, 'DeviceStatus': 'Inactive', 'AnomalyFlag': 1, 'AnomalyType': 'Overheat'}  
Sent row: {'OrderID': 'dc421056-af7a-402e-bdce-574f4abf8be6', 'CustomerID': 'CUST5925', 'Region': 'East', 'StoreID': 'Store001', 'Product': 'Phone', 'Quantity': 5, 'UnitPrice': 379.71, 'SalesAmount': 1898.55, 'OrderTime': '2025-08-25 10:50:41', 'DeviceType': 'SensorB', 'Temperature': 67.19, 'DeviceStatus': 'Active', 'AnomalyFlag': 0, 'AnomalyType': 'None'}  
Sent row: {'OrderID': '9e76b8cb-5bce-4f57-ad44-faa480222155', 'CustomerID': 'CUST2810', 'Region': 'North', 'StoreID': 'Store002', 'Product': 'Laptop', 'Quantity': 5, 'UnitPrice': 114.01, 'SalesAmount': 570.05, 'OrderTime': '2025-08-25 10:50:44', 'DeviceType': 'SensorC', 'Temperature': 59.5, 'DeviceStatus': 'Active', 'AnomalyFlag': 0, 'AnomalyType': 'None'}
```

15. Viewing events in Azure Event Hub to check whether events are being sent or not



16. Setting up Azure Event Hub connection in Databricks workspace using event hub connection string

```
# Event Hub connection parameters
eh_namespace = "stnamespace"
eh_name = "stevenhub"
eh_sas_key_name = "RootManageSharedAccessKey"
eh_sas_key = "2CAznsXvzxiVJsida8g21oNdeLAY3lsy+AEhD+s9Ss="

connection_string =
f"Endpoint=sb://{eh_namespace}.servicebus.windows.net/;SharedAccessKeyName={eh_sas_key_name};SharedAccessKey={eh_sas_key};EntityPath={eh_name}"
```

17. Modifying Bronze Layer Ingestion to capture real time streaming data

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType, IntegerType
from pyspark.sql import functions as F

# Define schema for Event Hub messages
event_schema = StructType([
    StructField("OrderID", StringType(), True),
    StructField("CustomerID", StringType(), True),
    StructField("Region", StringType(), True),
    StructField("StoreID", StringType(), True),
    StructField("Product", StringType(), True),
    StructField("Quantity", IntegerType(), True),
    StructField("UnitPrice", DoubleType(), True),
    StructField("SalesAmount", DoubleType(), True),
    StructField("OrderTime", StringType(), True),    # can cast to Timestamp later in
Silver
    StructField("DeviceType", StringType(), True),
    StructField("Temperature", DoubleType(), True),
```

```

    StructField("DeviceStatus", StringType(), True),
    StructField("AnomalyFlag", IntegerType(), True),
    StructField("AnomalyType", StringType(), True)
])

# Read streaming data from Event Hub
stream_df = (spark.readStream
    .format("eventhubs")
    .option("eventhubs.connectionString", connection_string)
    .load()
)

# Event Hub messages are in 'body' column as binary, decode them
decoded_df = (stream_df
    .withColumn("body_str", F.col("body").cast("string"))
    .select(F.from_json("body_str", event_schema).alias("data"))
    .select("data.*")
)

# Write stream to Bronze Delta table
bronze_stream = (decoded_df.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")
    .table("st_capstone_retail.bronze.events_bronze") # new bronze table for
streaming
)

```

18.

The screenshot shows a Databricks workspace with a notebook titled 'bronze_layer'. The notebook contains a Python script that attempts to stream data from Event Hubs into a Delta table. The script is as follows:

```
25 .option(eventhubs.connectionstring, connection_string)
26 .load()
27 )
28
29 # Event Hub messages are in 'body' column as binary, decode them
30 decoded_df = (stream_df
31   .withColumn("body_str", F.col("body").cast("string"))
32   .select(F.from_json("body_str", event_schema).alias("data"))
33   .select("data.*"))
34 )
35
36 # Write stream to Bronze Delta table
37 bronze_stream = (decoded_df.writeStream
38   .format("delta")
39   .outputMode("append")
40   .option("checkpointLocation", "/mnt/delta/events_bronze/_checkpoints/")
41   .table("st_capstone_retail.bronze.events_bronze") # new bronze table for streaming
42 )
43
```

The script fails with the following error message:

```
> [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on a shared cluster. SQLSTATE: 0A000
```

The error message indicates that the Event Hubs data source is not supported as a streaming source on a shared cluster. The user is prompted to 'Diagnose error' or 'Debug'.